REDES NEURONAIS ARTIFICIAIS

Luís Morgado

ISEL-DEI

APRENDIZAGEM EM REDES NEURONAIS

- Dois tipos principais de aprendizagem
  - **Supervisionada**
    - A aprendizagem supervisionada é um tipo de aprendizagem na qual a rede é treinada com um conjunto de dados etiquetados
      - Cada exemplo de treino possui uma entrada (por exemplo, uma imagem) e uma saída desejada (por exemplo, a classe da imagem)
    - A rede aprende associações entre entradas e saídas
  - **Não supervisionada**
    - A aprendizagem não supervisionada é um tipo de aprendizagem na qual a rede é treinada com um conjunto de dados não etiquetados
      - Os exemplos de treino não possuem a indicação da saída associada
    - A rede aprende padrões de regularidade dos dados

APRENDIZAGEM SUPERVISIONADA

- Na aprendizagem supervisionada, durante o treino a rede aprende a associar as entradas às saídas correspondentes
- O objetivo é que, após o treino, a rede seja capaz de prever para novas entradas a saída correta
- Exemplos de tarefas de aprendizagem supervisionada:
  - **Classificação**
    - Classificar uma imagem numa de várias categorias (por exemplo, pessoa, animal, objecto)
  - **Detecção de objetos**
    - Identificar e localizar objetos numa imagem

APRENDIZAGEM NÃO SUPERVISIONADA

- Na aprendizagem não supervisionada, o objetivo é descobrir padrões de regularidade nos dados, sem a necessidade de informação sobre as saídas esperadas
- A rede aprende representações de agrupamentos (*clusters*) de dados similares ou de características comuns nos dados
- Exemplos de tarefas de aprendizagem não supervisionada:
  - **Agrupamento (*clustering*)**
    - Organizar dados em agrupamentos com base nas suas semelhanças
  - **Redução de dimensionalidade**
    - Identificar as principais características dos dados e projetá-los num espaço de dimensão menor

APRENDIZAGEM EM REDES NEURONAIS

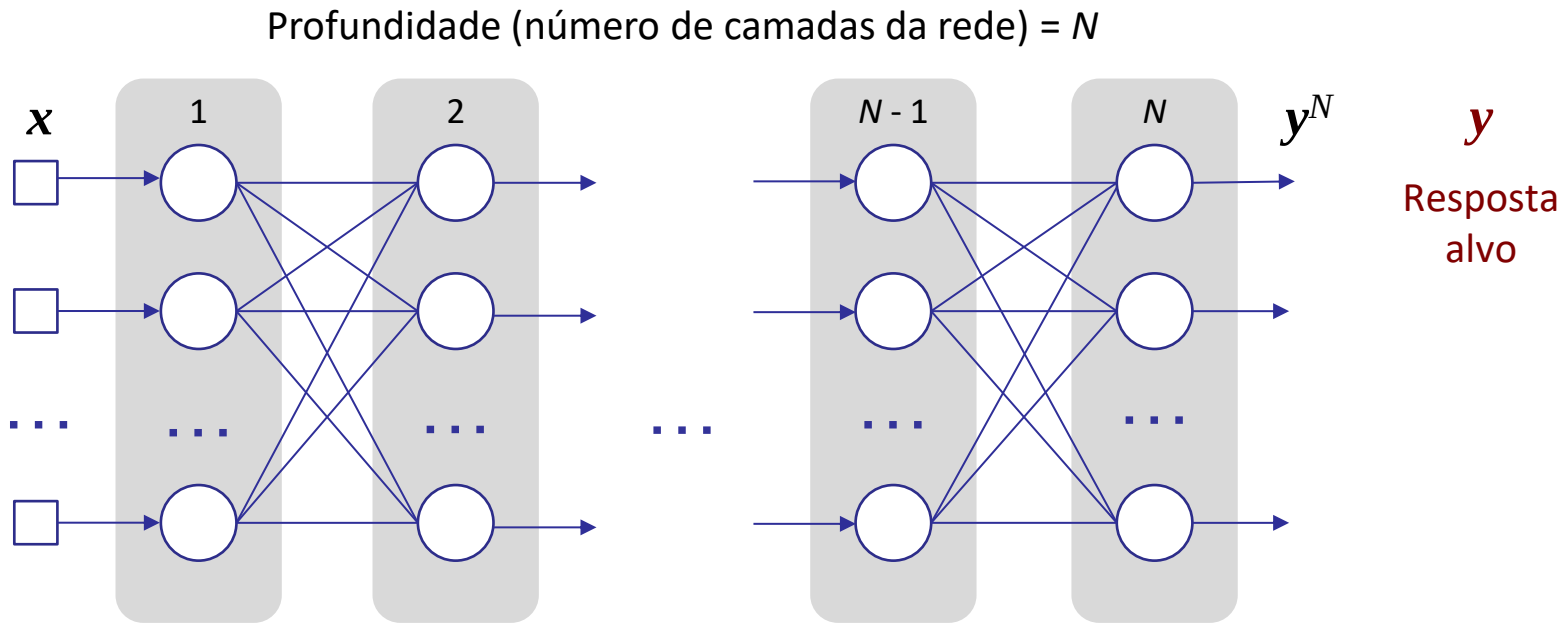
- **Aprendizagem "Hebbiana"**
  - A mais antiga regra de aprendizagem (Hebb, 1949)
  - A eficiência da sinapse aumenta se os neurónios em ambos os lados das sinapses forem activados
    - "*Cells that fire together, wire together*" (Löwel, 1992)

$$w_{ij}(t+1) = w_{ij}(t) + y_j(t)x_i(t)$$

  - **Viável para redes de uma única camada**
  - Aprendizagem local
    - A alteração dos pesos sinápticos depende apenas dos neurónios ligados à sinapse

APRENDIZAGEM EM REDES NEURONAIS

- **Aprendizagem por correcção do erro**
  - Aprendizagem supervisionada
  - Para cada estímulo de treino $\boldsymbol{x}$ e resposta $\boldsymbol{y}^N$ é indicada à rede a resposta alvo $\boldsymbol{y}$ correspondente



APRENDIZAGEM EM REDES NEURONAIS

- **Aprendizagem por correcção do erro**
  - A aprendizagem consiste na **minimização** de uma função de erro, ou perda (*loss*), $\boldsymbol{L}(\boldsymbol{y}, \boldsymbol{y}^N)$, entre a resposta $\boldsymbol{y}^N$ correspondente ao estímulo e a resposta $\boldsymbol{y}$ produzida pela rede
  - Com base nesse erro é realizada a alteração dos pesos sinápticos de forma a aproximar a resposta da rede à resposta desejada
  - **Aprendizagem por retropropagação do erro**
    - ***Algoritmo de Retropropagação***
      (Rumelhart, Hinton & Williams, 1986)

ALGORITMO DE RETROPROPAGAÇÃO

A aprendizagem com base no algoritmo de retropropagação pode ser descrita como um problema de optimização, no qual o objectivo é minimizar uma função de custo (*função de perda*) que avalia a divergência (*erro*) entre as previsões do modelo e os valores reais

Formulação como um problema de optimização

- O algoritmo de retropropagação tem por base o cálculo dos gradientes da função de perda necessários para actualizar os parâmetros do modelo (rede neuronal)
- A actualização dos parâmetros é realizada tendo por base um método de optimização como o método de *descida de gradiente*:
  - O conjunto de parâmetros θ do modelo é actualizado em função do gradiente da função de perda $L(\theta)$, no sentido oposto ao do gradiente (minimização):

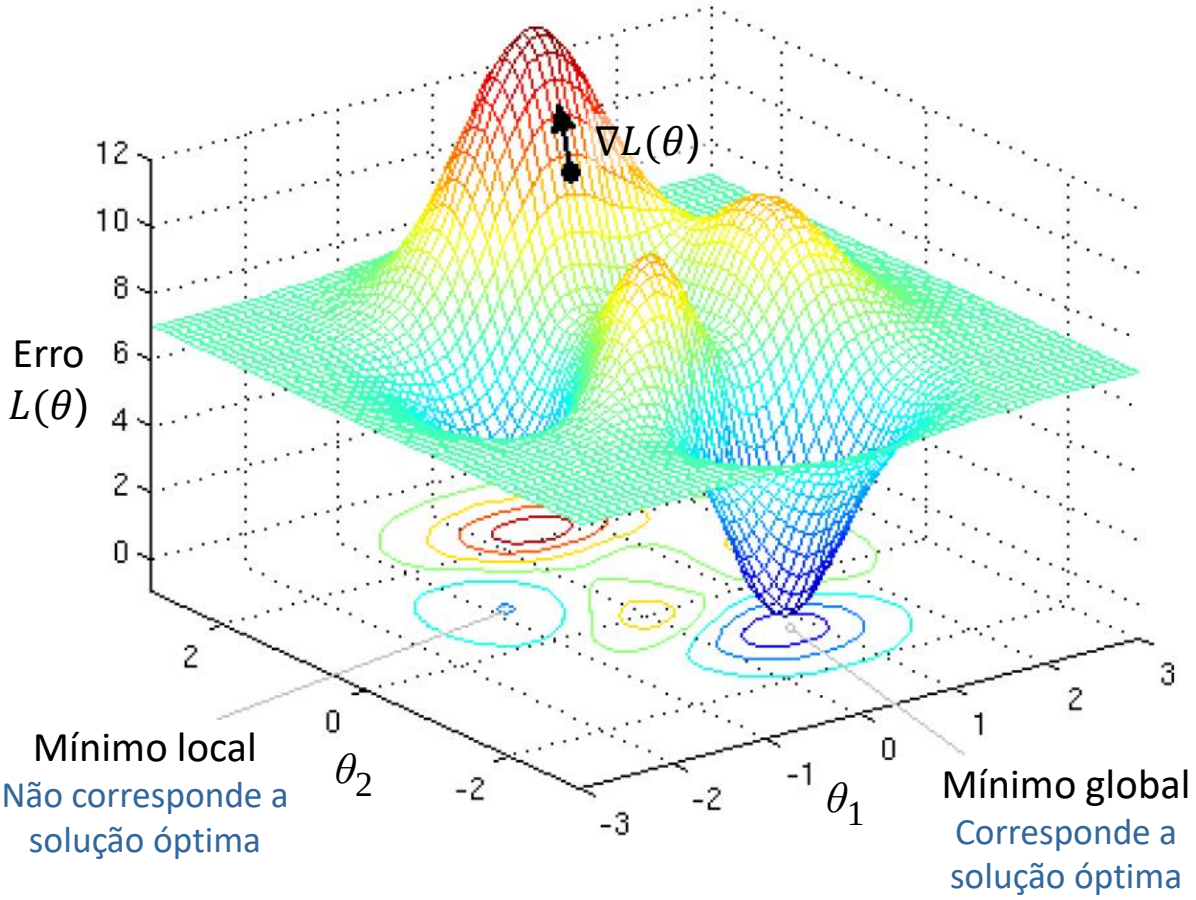
    $$\theta \leftarrow \theta - \alpha \nabla L(\theta)$$

  - Onde:
    - α é um hiper-parâmetro, designado *taxa de aprendizagem*, que determina a dimensão dos passos do algoritmo de optimização
    - $\nabla L(\theta)$ é o gradiente da função de perda $L(\theta)$

ALGORITMO DE RETROPROPAGAÇÃO

Optimização num espaço de parâmetros θ_i

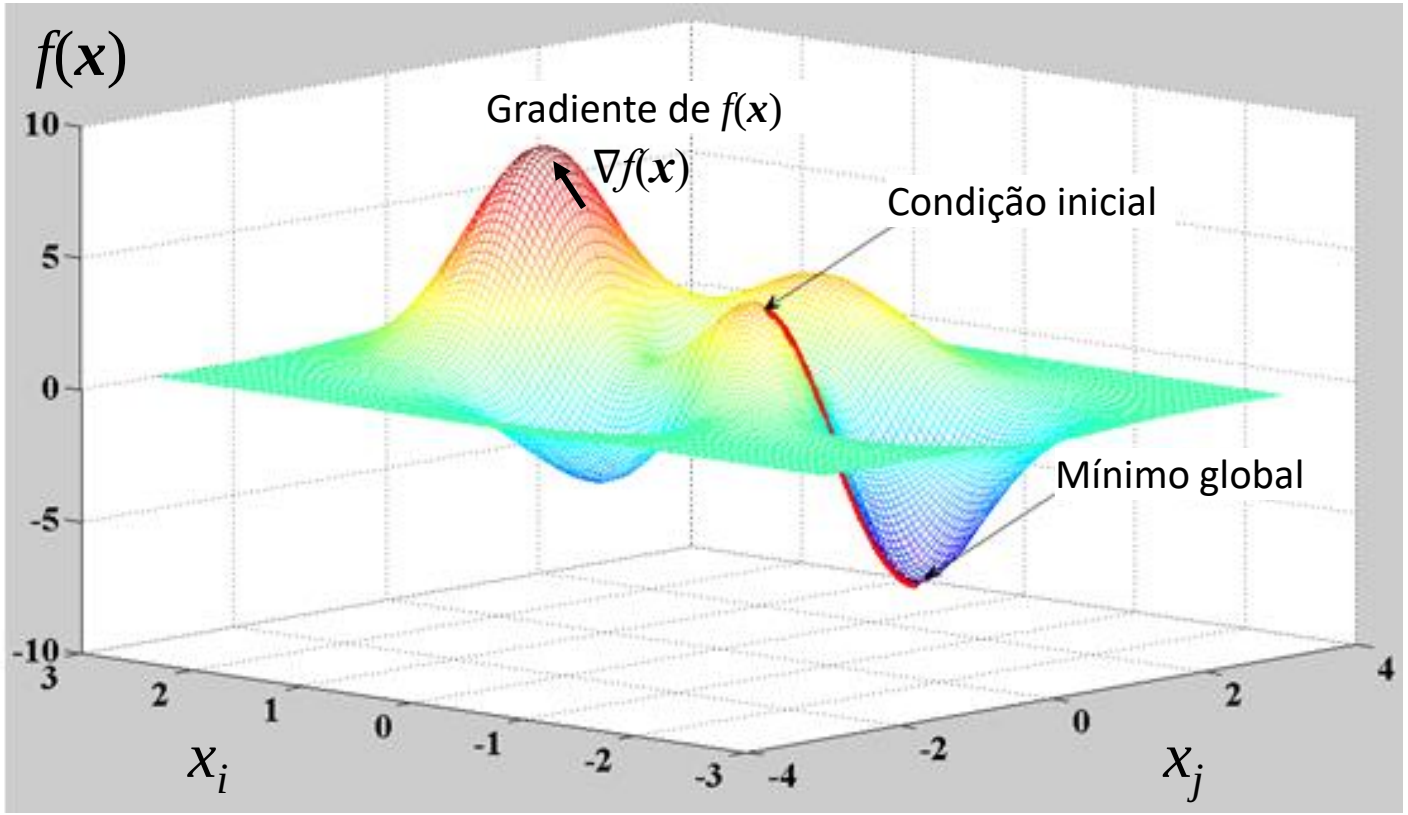
Minimização do erro de classificação



Adaptado de [Raizer *et al.*, 2009]

ALGORITMO DE DESCIDA DE GRADIENTE

Objectivo: Minimizar $f(x)$



$$\boldsymbol{x}^{n+1} = \boldsymbol{x}^n + \Delta\boldsymbol{x}^n$$

$$\Delta\boldsymbol{x}^n = -\alpha\nabla f(\boldsymbol{x})$$

$n \geq 0$ – Índice do passo

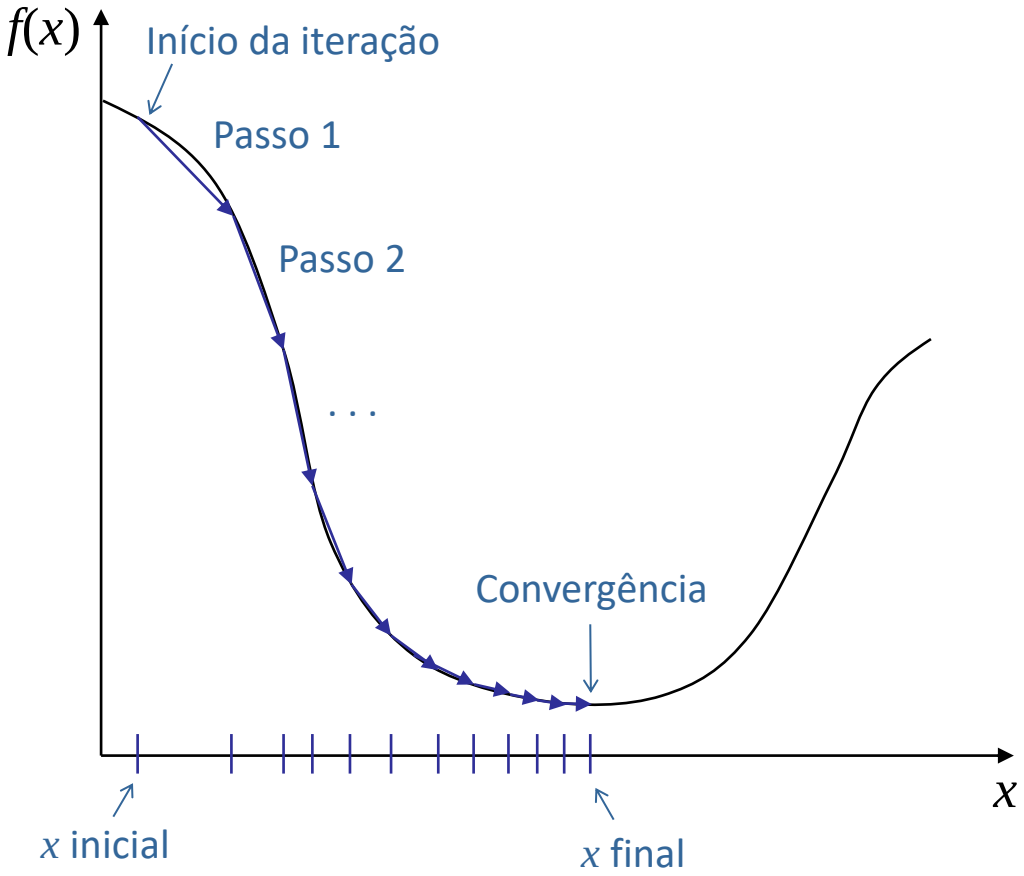
α – Dimensão do passo

$\Delta\boldsymbol{x}$ – Variação de $\boldsymbol{x}$ para actualização

ALGORITMO DE DESCIDA DE GRADIENTE

O valor de um parâmetro x de uma função $f(x)$ é actualizado iterativamente com base no gradiente da função, no sentido oposto ao do gradiente (minimização), até ser atingido um mínimo dessa função (convergência)

No entanto, o algoritmo de descida de gradiente, não dá garantias acerca do mínimo encontrado ser global ou local



ALGORITMO DE RETROPROPAGAÇÃO

- Aprendizagem supervisionada
- Dois conjuntos de dados representativos do domínio do problema
  - **Conjunto de treino**
    - Amostras para treino
  - **Conjunto de teste**
    - Amostras para teste
- Requer uma função de activação diferenciável

ALGORITMO DE RETROPROPAGAÇÃO

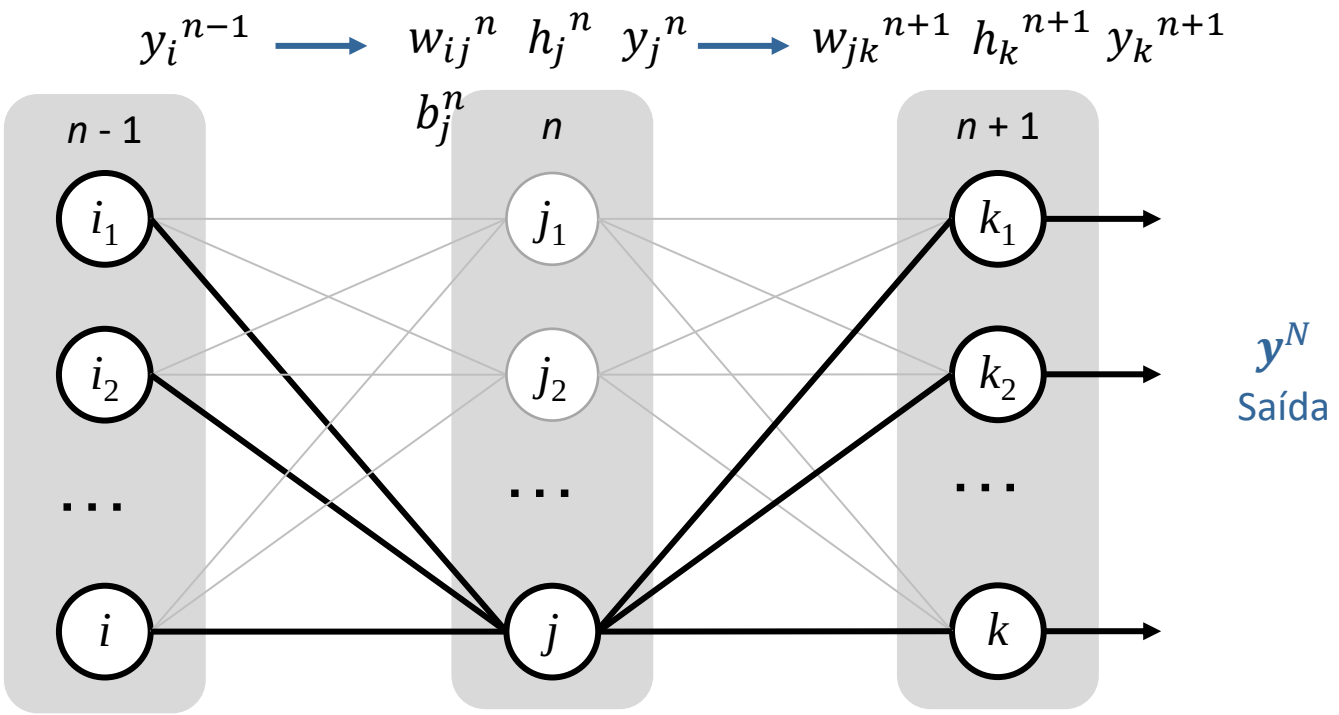
Organização geral do **algoritmo de aprendizagem por retropropagação do erro**:

- Repetir até ser atingido um critério de finalização do treino, por exemplo, erro mínimo obtido ou número máximo de passos de aprendizagem (iterações) atingido
  - Para cada exemplo $(\boldsymbol{x}, \boldsymbol{y})$:
    1. **Activação directa da rede**
       - Apresentar à entrada da rede o vector de entrada $\boldsymbol{x}$, activar sucessivamente as camadas da rede e observar o vector de saída $\boldsymbol{y}^N$
    2. **Retropropagação do erro**
       - Retropropagar o erro $\boldsymbol{L}(\boldsymbol{y}, \boldsymbol{y}^N)$ ao longo das sucessivas camadas da rede no sentido inverso (da saída para a entrada) actualizando os parâmetros de cada camada em função do erro

ALGORITMO DE RETROPROPAGAÇÃO

Propagação directa da entrada para gerar a saída

A activação directa da rede produz a saída da rede com base na entrada por activação sequencial encadeada das várias camadas da rede



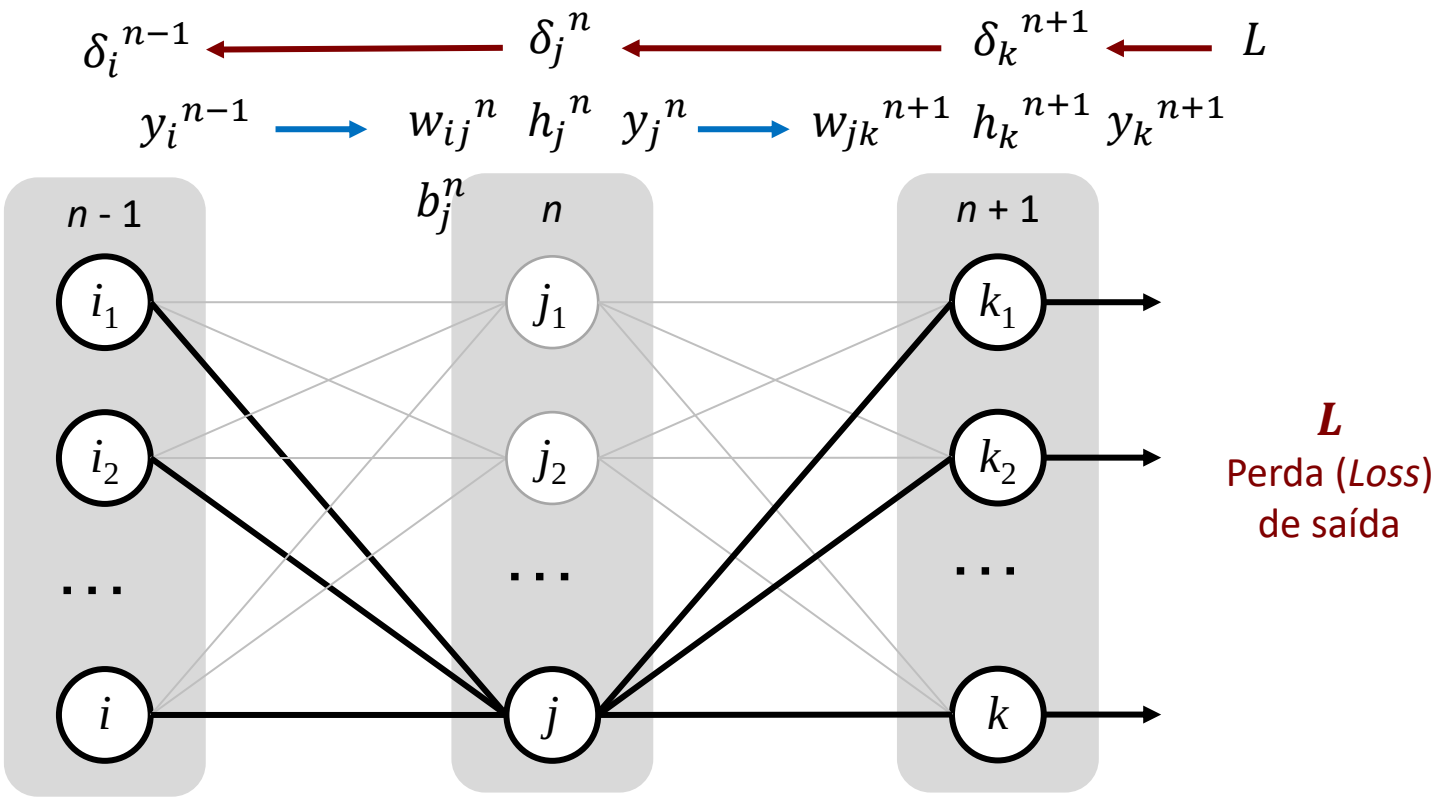
$$h_j^n = w_{ij}^n y_i^{n-1} + b_j^n$$

$$y_j^n = f\left(y_j^{n-1}\right) = \varphi^n\left(h_j^n\right)$$

ALGORITMO DE RETROPROPAGAÇÃO

Retropropagação do erro

Retropropagação do erro (*perda*) de saída para adaptação dos parâmetros da rede



$$h_j^{\,n} = w_{ij}^{\,n} y_i^{\,n-1} + b_j^n$$

$$y_j^{\,n} = f\left(y_j^{\,n-1}\right) = \varphi^n\left(h_j^{\,n}\right)$$

ALGORITMO DE RETROPROPAGAÇÃO

Objectivo:

- Minimizar o erro de previsão de uma rede neuronal

Método

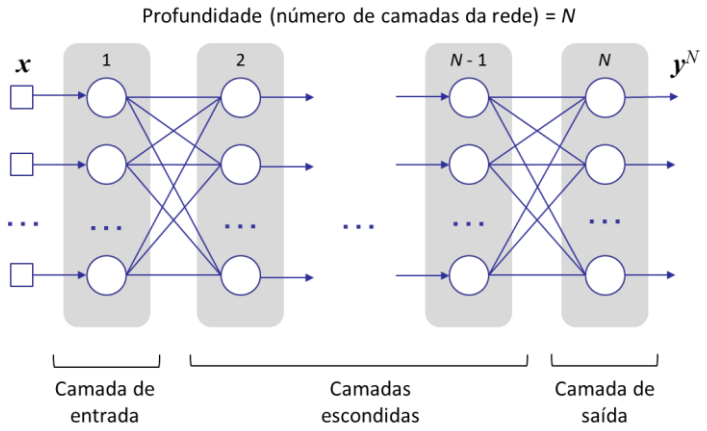
- Retropropagar o erro ao longo das várias camadas da rede, adaptando os parâmetros de cada camada de modo a minimizar o erro de previsão

Métrica de erro

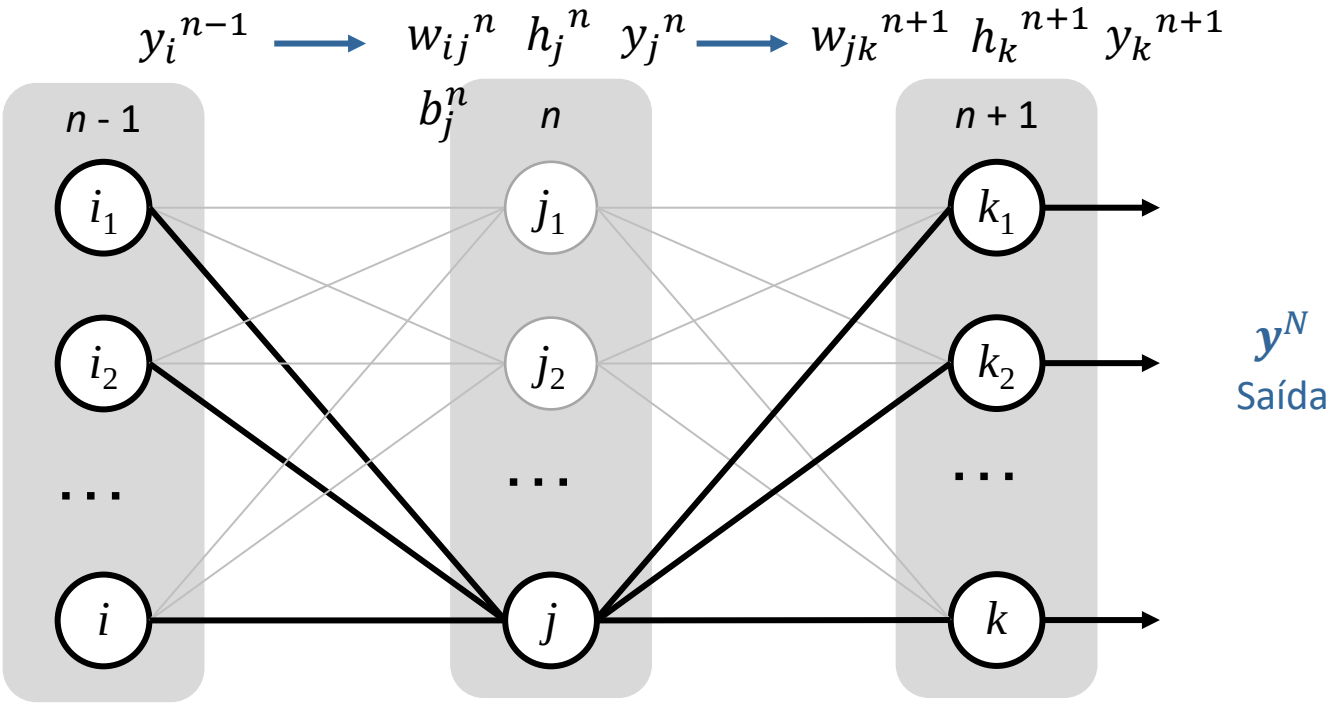
- Perda de classificação: $L(\boldsymbol{y}, \boldsymbol{y}^N)$

Parâmetros da rede a adaptar

- Rede com N camadas
- $\theta = \{\boldsymbol{W}^2, \boldsymbol{b}^2, \ldots, \boldsymbol{W}^{N-1}, \boldsymbol{b}^{N-1}, \boldsymbol{W}^N, \boldsymbol{b}^N\}$



ALGORITMO DE RETROPROPAGAÇÃO



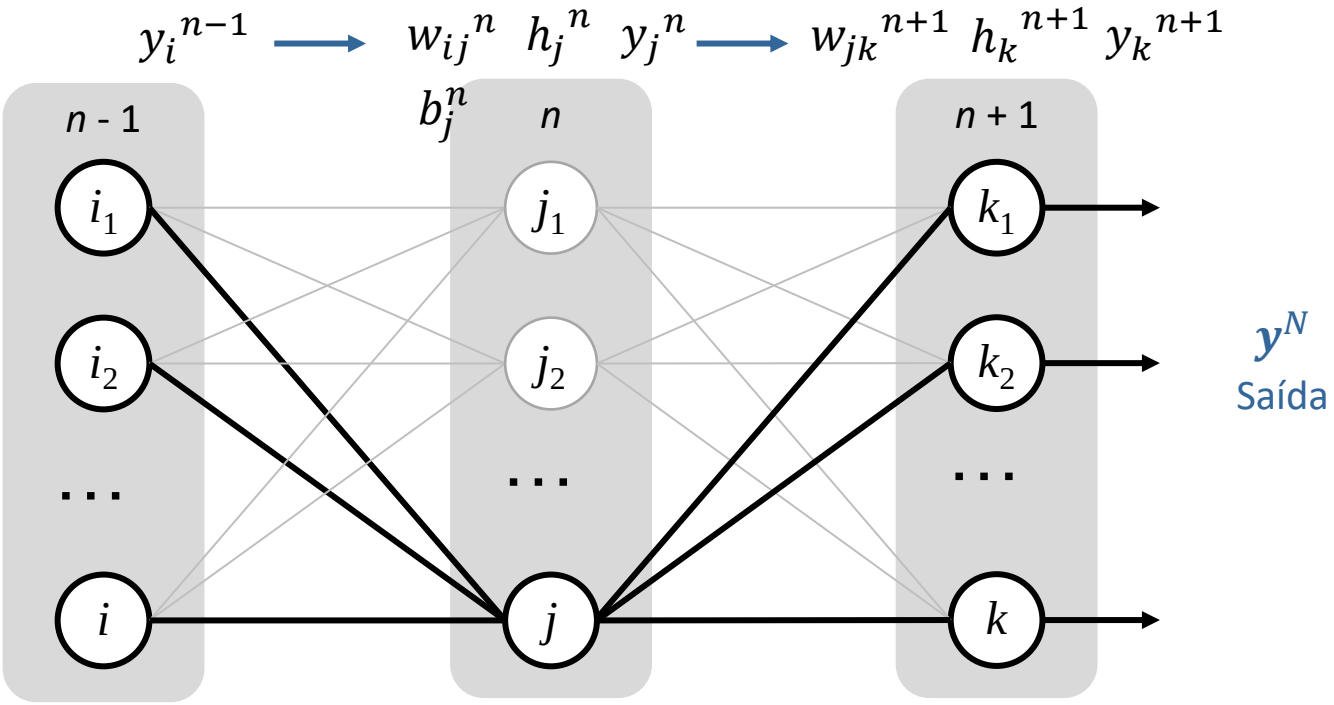
Consideremos um neurónio j de uma camada n de uma rede neuronal multicamada com neurónios índice i na camada anterior e neurónios índice k na camada seguinte

- Para cada peso w_{ij}^n e para cada pendor b_j^n do neurónio j pretende-se calcular as variações Δw_{ij}^n e Δb_j^n desse peso e pendor para realizar as respectiva actualizações

$$w_{ij}^n \leftarrow w_{ij}^n + \Delta w_{ij}^n$$

$$b_j^n \leftarrow b_j^n + \Delta b_j^n$$

ALGORITMO DE RETROPROPAGAÇÃO



Considerando o erro de saída da rede dado pela função de perda $L(\boldsymbol{y}, \boldsymbol{y}^N)$, as variações Δw_{ij}^n e Δb_j^n de peso e pendor de um neurónio j de uma camada n são dadas por:

$$\Delta w_{ij}^n = -\alpha \frac{\partial L}{\partial w_{ij}^n}$$

Variação do erro em relação ao peso
(α – Taxa de aprendizagem)

$$\Delta b_j^n = -\alpha \frac{\partial L}{\partial b_j^n}$$

Variação do erro em relação ao pendor

ALGORITMO DE RETROPROPAGAÇÃO

Actualização de pesos e pendores com base na retropropagação do erro de saída

α – Taxa de aprendizagem

$$\Delta w_{ij}^{n} = -\alpha \frac{\partial L}{\partial w_{ij}^{n}}$$

$$\Delta b_{j}^{n} = -\alpha \frac{\partial L}{\partial b_{j}^{n}}$$

$$w_{ij}^{n} \leftarrow w_{ij}^{n} + \Delta w_{ij}^{n}$$

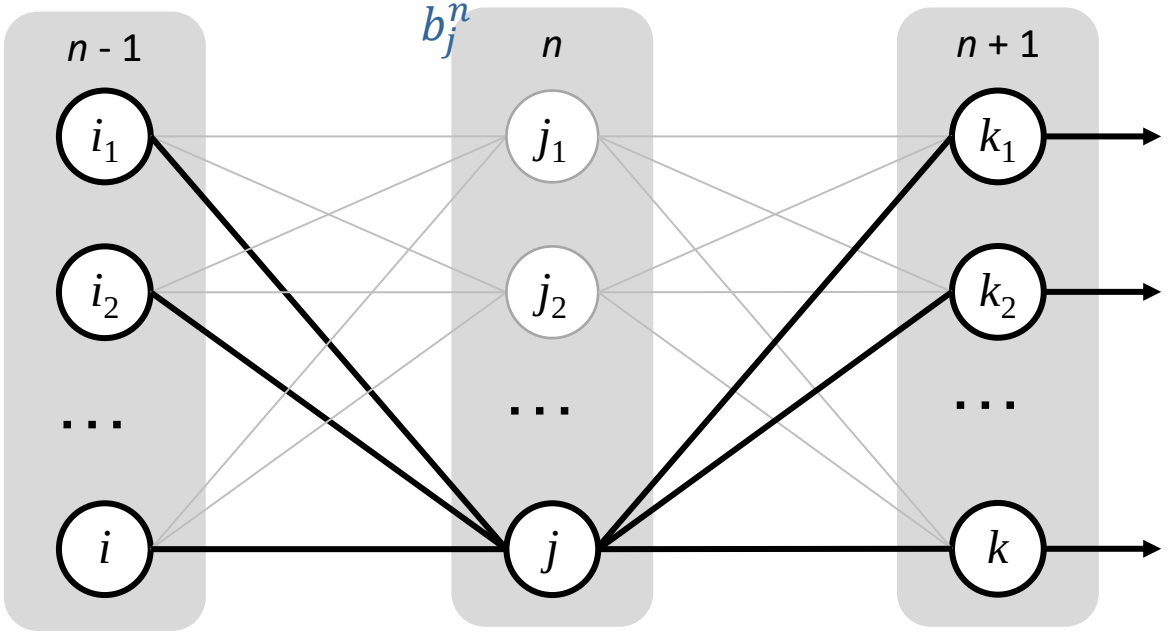
$$b_{j}^{n} \leftarrow b_{j}^{n} + \Delta b_{j}^{n}$$

Retropropagação do erro de saída

$\boldsymbol{L}$
Perda (*Loss*) de saída

y_i^{n-1} w_{ij}^{n} h_j^{n} y_j^{n} w_{jk}^{n+1} h_k^{n+1} y_k^{n+1} $\boldsymbol{y}$

b_j^{n}



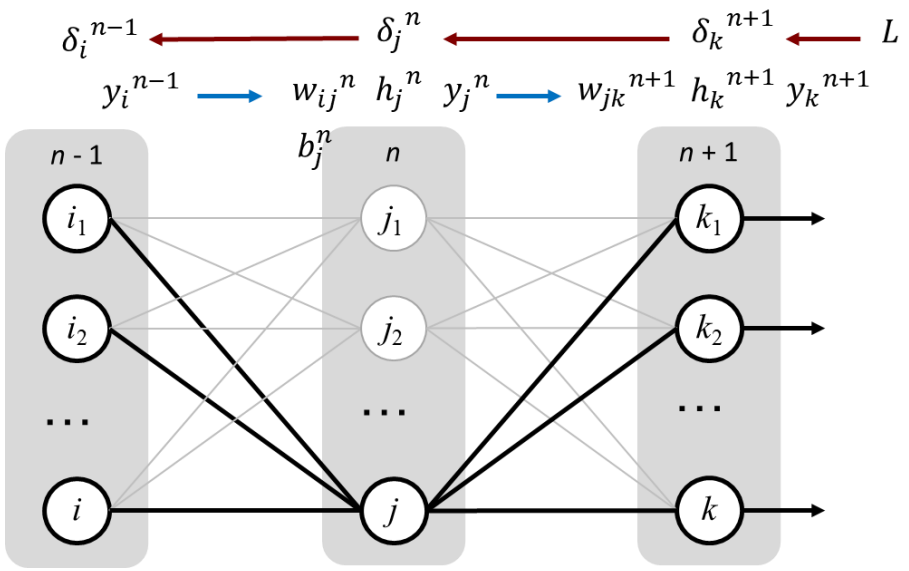
ALGORITMO DE RETROPROPAGAÇÃO

Cálculo da variação do erro em relação aos **pesos**

$$\Delta w_{ij}^{n} = -\alpha \frac{\partial L}{\partial w_{ij}^{n}}$$

α – Taxa de aprendizagem

$$\frac{\partial L}{\partial w_{ij}^{n}} = \frac{\partial L}{\partial y_{j}^{n}} \frac{\partial y_{j}^{n}}{\partial w_{ij}^{n}}$$



Regra de encadeamento de derivadas

$$\frac{dy}{dz} = \frac{\partial y}{\partial x}\frac{dx}{dz} = \frac{dx}{dz}\frac{\partial y}{\partial x}$$

ALGORITMO DE RETROPROPAGAÇÃO

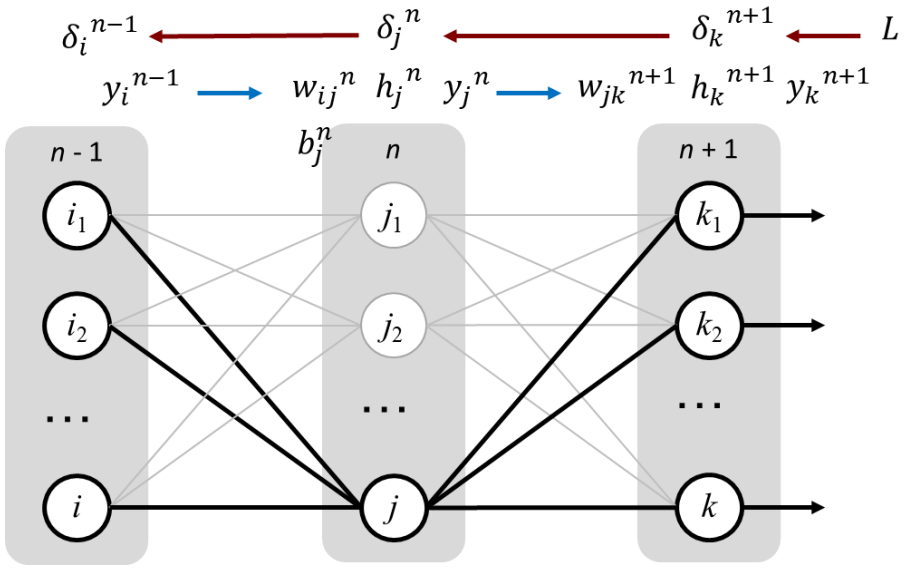
$$\frac{\partial L}{\partial w_{ij}^n} = \frac{\partial L}{\partial y_j^n}\frac{\partial y_j^n}{\partial w_{ij}^n}$$

$$\frac{\partial L}{\partial w_{ij}^n} = \frac{\partial L}{\partial y_j^n}\frac{\partial y_j^n}{\partial h_j^n}\frac{\partial h_j^n}{\partial w_{ij}^n}$$

$$\frac{\partial L}{\partial w_{ij}^n} = \frac{\partial L}{\partial y_j^n}\frac{\partial y_j^n}{\partial h_j^n} y_i^{n-1}$$

$$\delta_j^n = \frac{\partial L}{\partial y_j^n}$$

$$\boxed{\frac{\partial L}{\partial w_{ij}^n} = \delta_j^n \frac{\partial y_j^n}{\partial h_j^n} y_i^{n-1}}$$



δ_j^n - Componente de propagação do erro

(Variação do erro em relação à saída de cada camada)

ALGORITMO DE RETROPROPAGAÇÃO

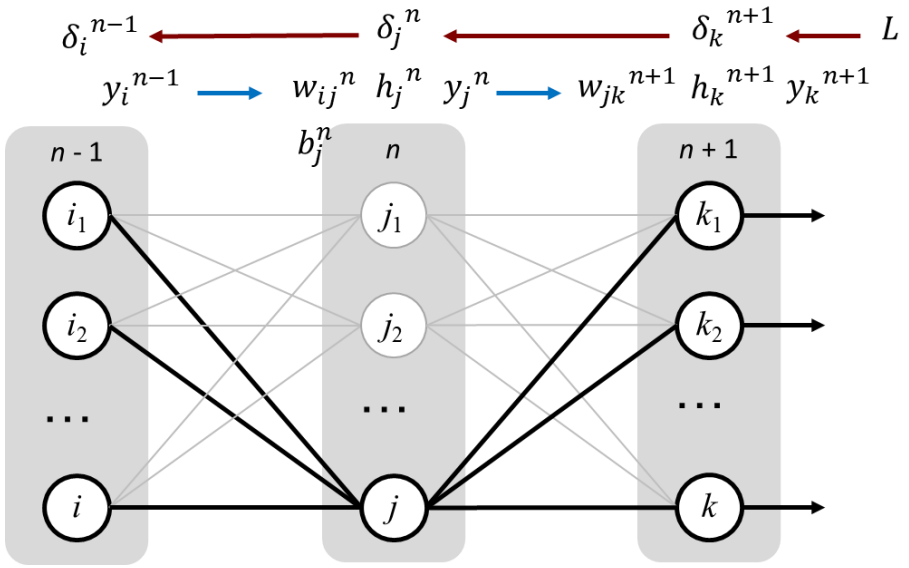
$$\delta_j^n = \frac{\partial L}{\partial y_j^n}$$

$$\delta_j^n = \frac{\partial L}{\partial y_k^{n+1}} \frac{\partial y_k^{n+1}}{\partial y_j^n}$$

$$\delta_j^n = \frac{\partial L}{\partial y_k^{n+1}} \frac{\partial y_k^{n+1}}{\partial h_k^{n+1}} \frac{\partial h_k^{n+1}}{\partial y_j^n}$$

$$\delta_j^n = \delta_k^{n+1} \frac{\partial y_k^{n+1}}{\partial h_k^{n+1}} \frac{\partial h_k^{n+1}}{\partial y_j^n}$$

$$\delta_j^n = \frac{\partial h_k^{n+1}}{\partial y_j^n} \delta_k^{n+1} \frac{\partial y_k^{n+1}}{\partial h_k^{n+1}}$$



δ_j^n - Componente de propagação do erro

(Variação do erro em relação à saída de cada camada)

ALGORITMO DE RETROPROPAGAÇÃO

$$\delta_j^n = \frac{\partial h_k^{n+1}}{\partial y_j^n} \delta_k^{n+1} \frac{\partial y_k^{n+1}}{\partial h_k^{n+1}}$$

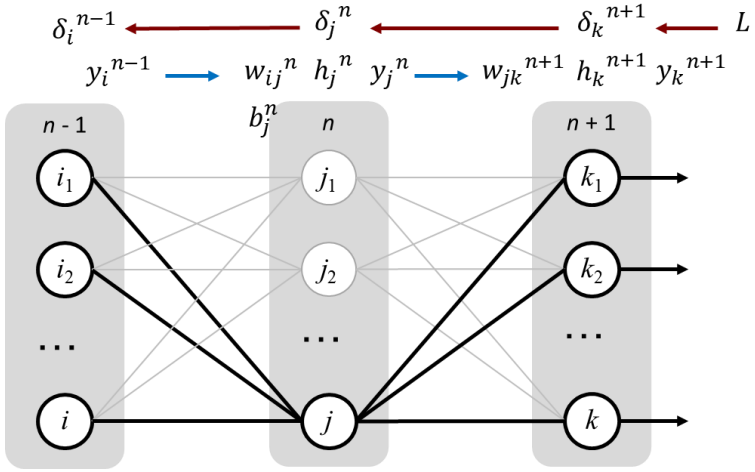
$$\delta_j^n = \frac{\partial \left(\sum_k y_j^n w_{jk}^{n+1} + b_k^{n+1} \right)}{\partial y_j^n} \delta_k^{n+1} \frac{\partial y_k^{n+1}}{\partial h_k^{n+1}}$$

$$\delta_j^n = \sum_k w_{jk}^{n+1} \delta_k^{n+1} \frac{\partial y_k^{n+1}}{\partial h_k^{n+1}}$$

$$y'^{n+1}_k = \frac{\partial y_k^{n+1}}{\partial h_k^{n+1}}$$

$$\boxed{\delta_j^n = \sum_k w_{jk}^{n+1} \delta_k^{n+1} y'^{n+1}_k}$$

$$\boxed{\delta_i^{n-1} = \sum_j w_{ij}^n \delta_j^n y'^n_j}$$



δ_j^n - Componente de propagação do erro

(Variação do erro em relação à saída de cada camada)

ALGORITMO DE RETROPROPAGAÇÃO

Cálculo da variação do erro em relação aos **pesos**

$$\Delta w_{ij}^{n} = -\alpha \frac{\partial L}{\partial w_{ij}^{n}}$$

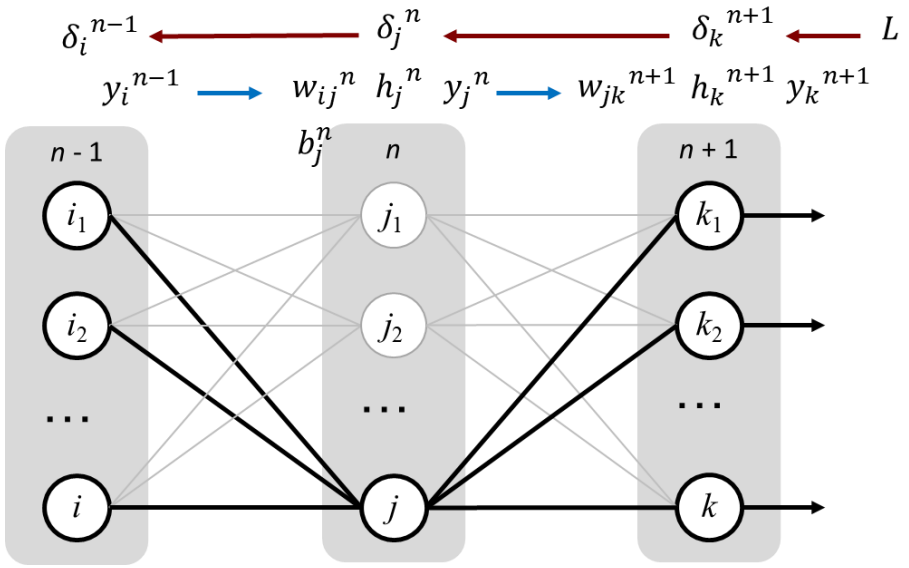
α – Taxa de aprendizagem

$$\frac{\partial L}{\partial w_{ij}^{n}} = \delta_j^n \frac{\partial y_j^n}{\partial h_j^n} y_i^{n-1}$$

$$\Delta w_{ij}^{n} = -\alpha \delta_j^n \frac{\partial y_j^n}{\partial h_j^n} y_i^{n-1}$$

$$y'^{n}_{j} = \frac{\partial y_j^n}{\partial h_j^n}$$

$$\boxed{\Delta w_{ij}^{n} = -\alpha \delta_j^n y'^{n}_{j} y_i^{n-1}}$$



δ_j^n - Componente de propagação do erro

(Variação do erro em relação à saída de cada camada)

ALGORITMO DE RETROPROPAGAÇÃO

Cálculo da variação do erro em relação aos **pendores**

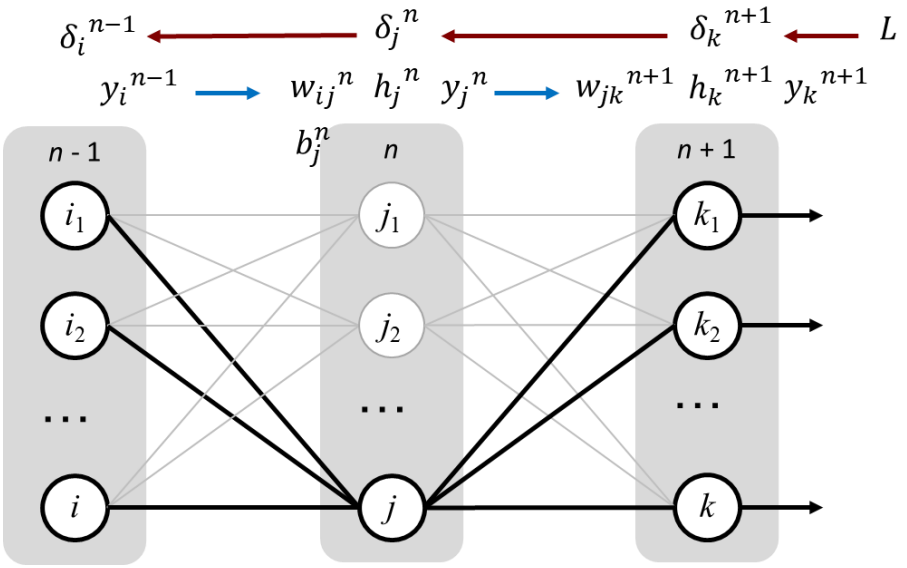
$$\frac{\partial L}{\partial b_j^n} = \frac{\partial L}{\partial y_j^n}\frac{\partial y_j^n}{\partial b_j^n}$$

$$\frac{\partial L}{\partial b_j^n} = \frac{\partial L}{\partial y_j^n}\frac{\partial y_j^n}{\partial h_j^n}\frac{\partial h_j^n}{\partial b_j^n}$$

$$\delta_j^n = \frac{\partial L}{\partial y_j^n}$$

$$\frac{\partial h_j^n}{\partial b_j^n} = \frac{\partial\left(\sum_i y_i^{n-1} w_{ij}^n + b_j^n\right)}{\partial b_j^n} = 1$$

$$\boxed{\frac{\partial L}{\partial b_j^n} = \delta_j^n \frac{\partial y_j^n}{\partial h_j^n}}$$



δ_j^n - Componente de propagação do erro

(Variação do erro em relação à saída de cada camada)

ALGORITMO DE RETROPROPAGAÇÃO

Cálculo da variação do erro em relação aos **pendores**

$$\Delta b_j^n = -\alpha \frac{\partial L}{\partial b_j^n}$$

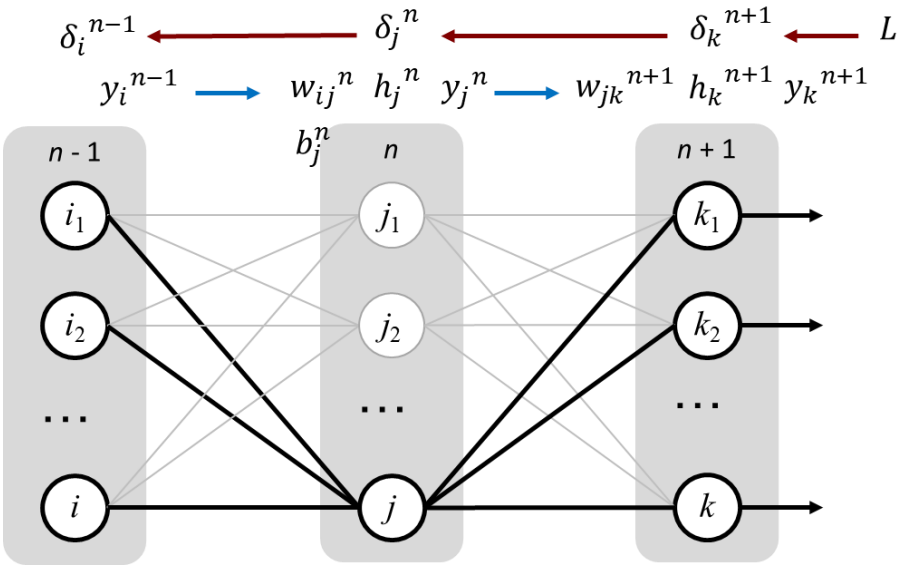
α – Taxa de aprendizagem

$$\frac{\partial L}{\partial b_j^n} = \delta_j^n \frac{\partial y_j^n}{\partial h_j^n}$$

$$\Delta b_j^n = -\alpha \delta_j^n \frac{\partial y_j^n}{\partial h_j^n}$$

$$y'^n_j = \frac{\partial y_j^n}{\partial h_j^n}$$

$$\boxed{\Delta b_j^n = -\alpha \delta_j^n y'^n_j}$$



δ_j^n - Componente de propagação do erro

(Variação do erro em relação à saída de cada camada)

ALGORITMO DE RETROPROPAGAÇÃO

Camada de saída

A componente de propagação do erro δ_j^n da camada de saída é dependente da função de perda definida $L(\boldsymbol{y}, \boldsymbol{y}^N)$

Considerando a função de perda definida pelo *Erro Quadrático Médio* (EQM):

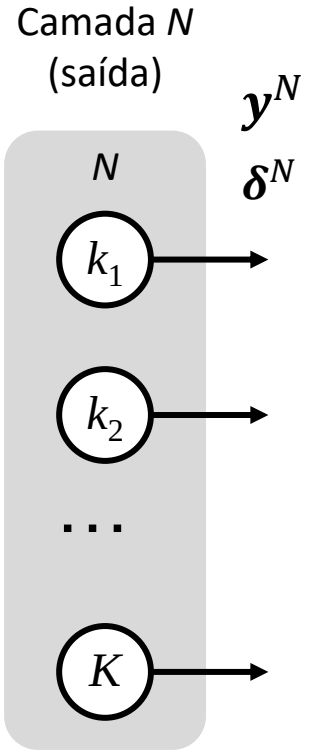
$$EQM(\boldsymbol{y}^N) = \frac{1}{K}\sum_{k=1}^{K}(y_k^N - y_k)^2$$

y_k^N – Valor estimado da saída *k*

y_k – Valor a aprender para a saída *k*

Para simplificação de cálculos, a função de perda é normalmente definida com um factor de normalização constante

$$L(\boldsymbol{y}, \boldsymbol{y}^N) = \frac{K}{2}\frac{1}{K}\sum_{k=1}^{K}(y_k^N - y_k)^2$$



ALGORITMO DE RETROPROPAGAÇÃO

Camada de saída

A componente de propagação do erro δ_j^n da camada de saída é dependente da função de perda definida $L(\boldsymbol{y}, \boldsymbol{y}^N)$

Considerando a função de perda definida anteriormente:

$$L(\boldsymbol{y}, \boldsymbol{y}^N) = \frac{K}{2}\frac{1}{K}\sum_{k=1}^{K}(y_k^N - y_k)^2$$

$$\delta_k^N = \frac{\partial L}{\partial y_k^N}$$

$$\frac{\partial L}{\partial y_k^N} = \frac{K}{2}\frac{2}{K}(y_k^N - y_k)$$

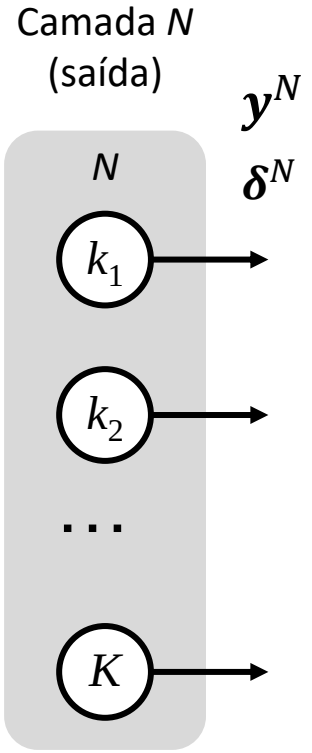
$$\delta_k^N = y_k^N - y_k$$

y_k^N – Valor estimado da saída *k*

y_k – Valor a aprender para a saída *k*

$\boldsymbol{y}^N$ – Vector de saída gerado pela rede

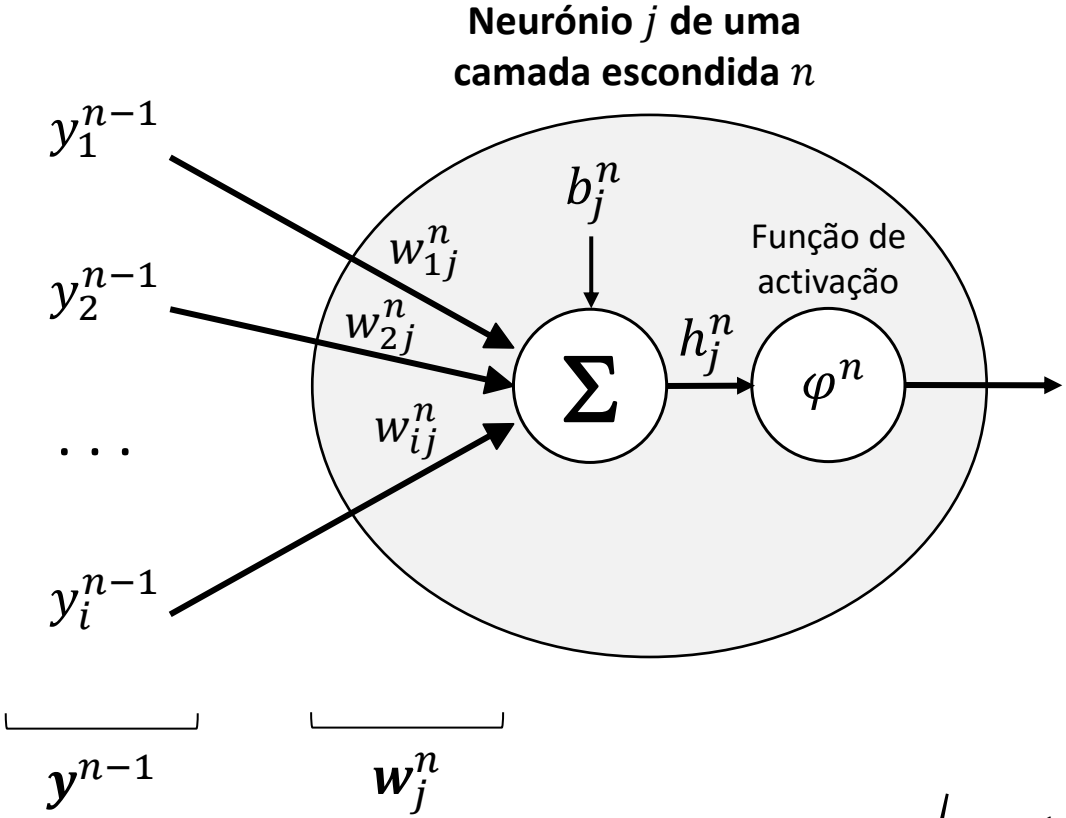
$\boldsymbol{\delta}^N$ – Vector de variação de erro de saída



ALGORITMO DE RETROPROPAGAÇÃO

Camada escondida

Adaptação dos pesos nas camadas escondidas



$$\Delta w_{ij}^n = -\alpha \delta_j^n y'^n_j y_i^{n-1}$$

$$y_j^n = \varphi^n(h_j^n)$$

$$y'^n_j = \frac{\partial y_j^n}{\partial h_j^n}$$

$$h_j^n = \left\langle \boldsymbol{y}^{n-1}, \boldsymbol{w}_j^n \right\rangle + b_j^n = \sum_{i=1}^{I} y_i^{n-1} w_{ij}^n + b_j^n$$

DERIVADA DA FUNÇÃO DE ACTIVAÇÃO

Exemplo: Função de activação sigmóide

$$\varphi(h) = \frac{1}{1+e^{-h}}$$

$$y = \varphi(h)$$

$$y' = \frac{\partial y}{\partial h} = \frac{\partial \varphi(h)}{\partial h}$$

$$= \frac{d\left(\frac{1}{1+e^{-h}}\right)}{dh}$$

$$= \frac{d\left(1+e^{-h}\right)^{-1}}{dh}$$

$$= \left(1+e^{-h}\right)^{-2} e^{-h}$$

$$= \frac{e^{-h}}{(1+e^{-h})^2}$$

$$= \frac{1}{1+e^{-h}} \frac{e^{-h}}{1+e^{-h}}$$

$$= \frac{1}{1+e^{-h}} \frac{1+e^{-h}-1}{1+e^{-h}}$$

$$= \frac{1}{1+e^{-h}} \left(\frac{1+e^{-h}}{1+e^{-h}} - \frac{1}{1+e^{-h}}\right)$$

$$= \frac{1}{1+e^{-h}} \left(1 - \frac{1}{1+e^{-h}}\right)$$

$$= \varphi(h)\left(1-\varphi(h)\right)$$

$$= y(1-y)$$

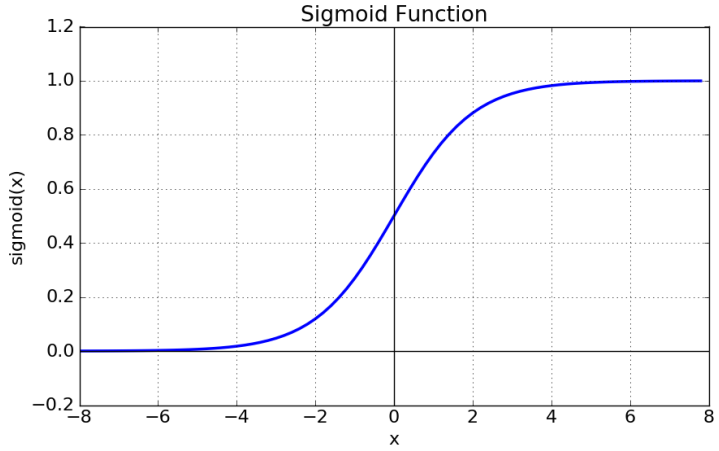
EXEMPLOS DE FUNÇÃO DE ACTIVAÇÃO

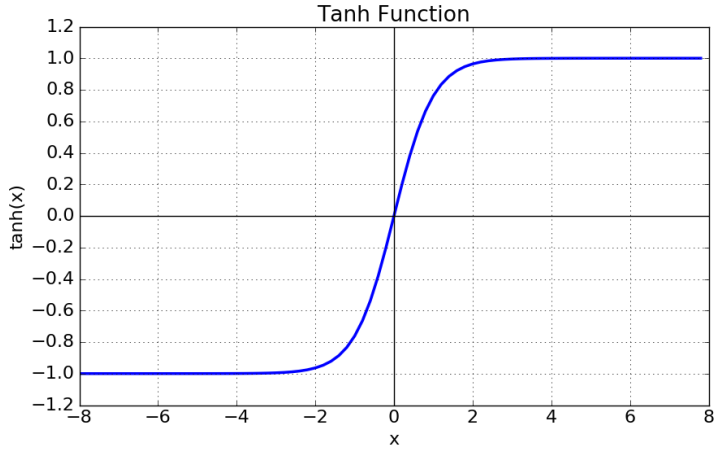
$$\varphi(h) = \frac{1}{1 + e^{-h}}$$

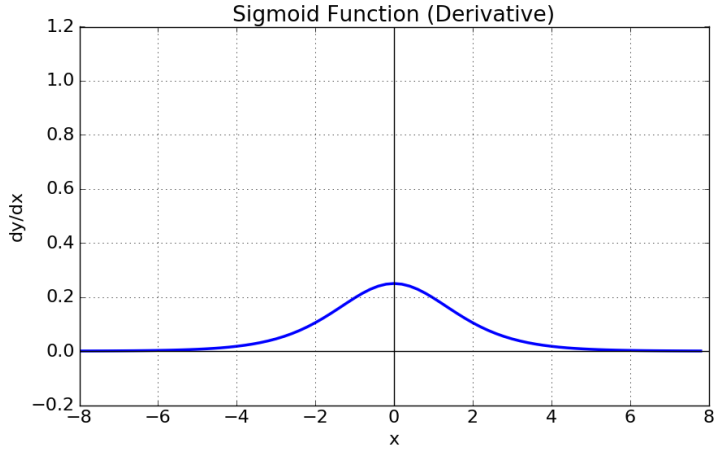
$$\varphi'(h) = \varphi(h)\big(1 - \varphi(h)\big)$$

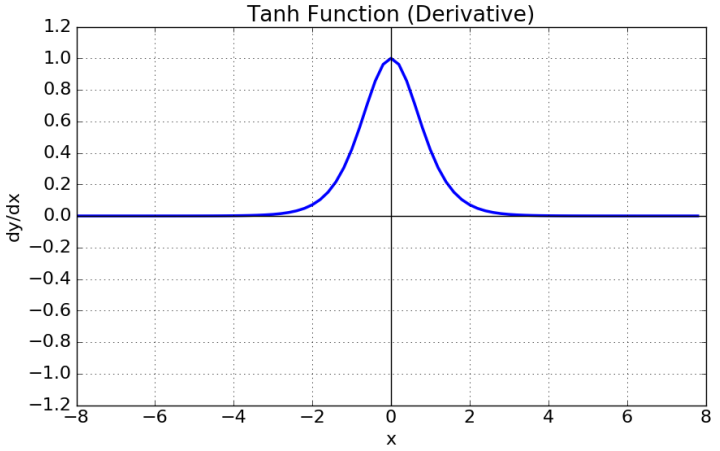
$$\varphi(h) = \tanh(h)$$

$$\varphi'(h) = 1 - \tanh(h)^2$$









ALGORITMO DE RETROPROPAGAÇÃO

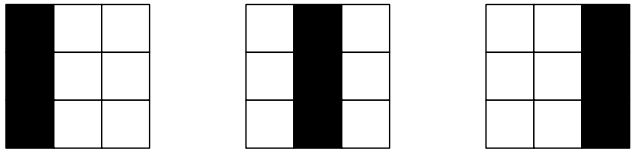
- **Iniciar** os parâmetros (pesos e pendores) da rede neuronal com valores aleatórios
- **Repetir** para várias iterações (*épocas*) de treino até que o erro seja suficientemente pequeno ou até que um número máximo de épocas seja alcançado

1. **Propagação directa**:
   - Para cada vector de entrada, calcular a saída da rede, activando sucessivamente as várias camadas da rede no sentido directo (da entrada para a saída)
2. **Cálculo do erro**:
   - Após obter a saída da rede, calcular o erro (*perda*), que expressa uma métrica de divergência entre a saída gerada pela rede (*previsão*) e a saída real
3. **Retropropagação do erro:**
   - Para a camada de saída e para cada camada escondida, calcular os componentes de propagação do erro e retropropagar o erro no sentido inverso (da saída para a entrada)
4. **Actualização dos parâmetros da rede:**
   - Com base na retropropagação do erro, actualizar os parâmetros (pesos e pendores) de cada camada da rede neuronal

ALGORITMO DE RETROPROPAGAÇÃO

Exemplo: Tarefa de reconhecimento de barras verticais

Pretende-se reconhecer barras verticais numa imagem de 3x3 pontos



Dimensionamento da rede

Número de camadas escondidas: 1
Dimensão da camada de entrada: 9 (Uma entrada por cada ponto da imagem)
Dimensão da camada escondida: 3
Dimensão da camada de saída: 1

Forma da rede = [9, 3, 1]
α = 0.2
Épocas = 1000
Erro máximo = 0.05
Função de activação = tanh

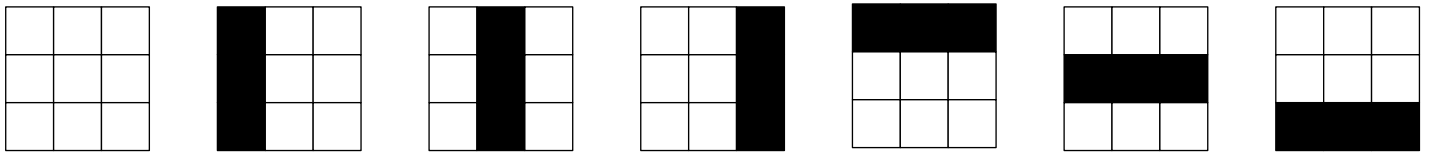
A dimensão de uma camada define o número de neurónios dessa camada

A tarefa a realizar é de reconhecimento (sim/não existe barra na imagem), pelo que apenas é necessário um neurónio de saída

ALGORITMO DE RETROPROPAGAÇÃO

Exemplo: Tarefa de reconhecimento de barras verticais

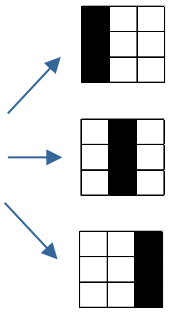
Exemplos de treino



Saída alvo: 0 1 1 1 0 0 0

Codificação dos dados de treino

entradas = [
[0, 0, 0, 0, 0, 0, 0, 0, 0],
[1, 0, 0, 1, 0, 0, 1, 0, 0],
[0, 1, 0, 0, 1, 0, 0, 1, 0],
[0, 0, 1, 0, 0, 1, 0, 0, 1],
[1, 1, 1, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 1, 1, 1, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 1, 1, 1]
]



saídas = [[0], [1], [1], [1], [0], [0], [0]]

Dimensão da camada de entrada: 9
(Uma entrada por cada ponto da imagem)

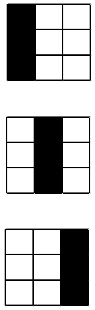
Dimensão da camada de saída: 1

ALGORITMO DE RETROPROPAGAÇÃO

Exemplo: Tarefa de reconhecimento de barras verticais

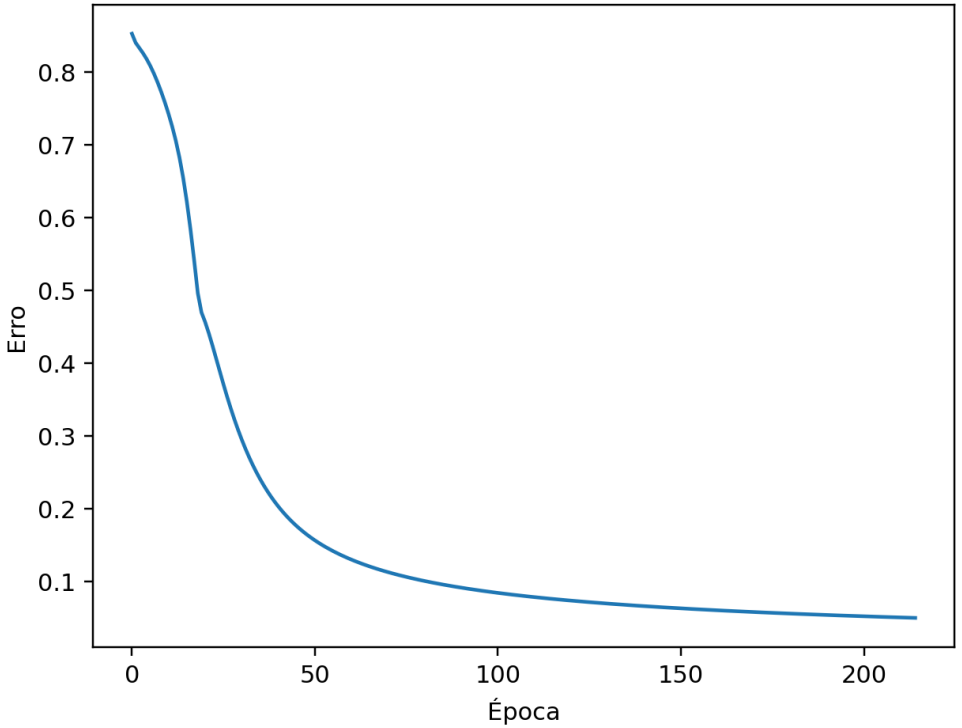
Dados de treino

entradas = [
[0, 0, 0, 0, 0, 0, 0, 0, 0],
[1, 0, 0, 1, 0, 0, 1, 0, 0],
[0, 1, 0, 0, 1, 0, 0, 1, 0],
[0, 0, 1, 0, 0, 1, 0, 0, 1],
[1, 1, 1, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 1, 1, 1, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 1, 1, 1]
]



saídas = [[0], [1], [1], [1], [0], [0], [0]]

Evolução do erro ao longo das épocas de treino



Resultados

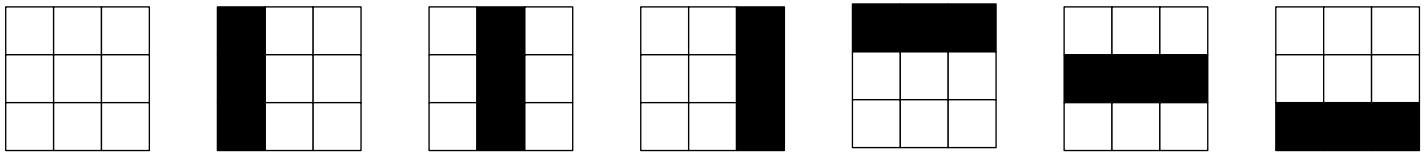
erro: 0.049937
época: 214

saídas = [[0.00039], [0.96084], [0.97566], [0.95017], [-0.0035], [-0.00469], [0.00007]]

ALGORITMO DE RETROPROPAGAÇÃO

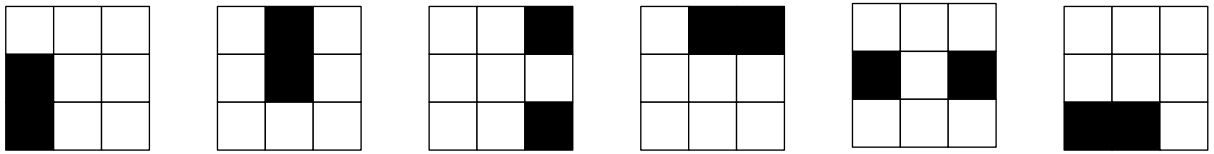
Exemplo: Tarefa de reconhecimento de barras verticais

Dados de treino



Saída alvo	0.00	1.00	1.00	1.00	0.00	0.00	0.00
Saída da rede	0.00	0.96	0.98	0.95	0.00	0.00	0.00

Dados de teste – Verificação da capacidade de generalização da rede neuronal



Saída da rede	0.98	0.96	0.84	0.62	-0.42	0.27

NEURÓNIO - IMPLEMENTAÇÃO

Informação associada a um neurónio e respectivos valores iniciais

1.	$\text{iniciar}(d, \varphi)$:	
2.	$d \in Z^+$	Dimensão de entrada (número ligações de entrada)
3.	$\varphi: R \rightarrow R$	Função de activação
4.	$\boldsymbol{w} = \text{random}(d, -1, 1)$	Vector de pesos das ligações de entrada
5.	$b = \text{random}(-1, 1)$	Pendor interno do neurónio
6.	$h = 0$	Soma da activação das entradas
7.	$y = 0$	Saída do neurónio
8.	$y' = 0$	**Derivada da saída do neurónio**

NEURÓNIO

Propagar entradas gerando a saída do neurónio e a respectiva derivada

1. propagar($\boldsymbol{x}$): — $\boldsymbol{x}$ - vector de entradas
2. $h = \boldsymbol{w} \cdot \boldsymbol{x} + b$
3. $y = \varphi(h)$
4. $y' = \frac{d\varphi(h)}{dh}$
5. return y

Produto escalar

$$\boldsymbol{a} \cdot \boldsymbol{b} = \sum_{i=1}^{n} a_i b_i = a_1 b_1 + a_2 b_2 + \ldots + a_n b_n$$

NEURÓNIO

Adaptar pesos das ligações de entrada e pendor interno

1. adaptar$(\delta, \boldsymbol{y}^{n-1}, \alpha)$:
2. $\Delta_{\boldsymbol{w}} = -\alpha y' \delta \cdot \boldsymbol{y}^{n-1}$ — Variação dos pesos
3. $\boldsymbol{w} = \boldsymbol{w} + \Delta_{\boldsymbol{w}}$ — Actualização dos pesos
4. $\Delta_b = -\alpha y' \delta$ — Variação do pendor
5. $b = b + \Delta_b$ — Actualização do pendor

δ Componente de propagação do erro de saída do neurónio

$\boldsymbol{y}^{n-1}$ Vector de saída da camada anterior $n-1$

α Taxa de aprendizagem

CAMADA DENSA

Adaptar pesos e pendores dos neurónios da camada

1. adaptar$\left(\boldsymbol{\delta}^n, \boldsymbol{y}^{n-1}, \alpha\right)$:
2. for $j = 1$ to d_s:
3. $neuronios[j].adaptar\left(\delta_j^n, \boldsymbol{y}^{n-1}, \alpha\right)$

$\boldsymbol{\delta}^n$	Vector de erro de saída da camada n
δ_j^n	Componente j do vector de erro de saída da camada n
$\boldsymbol{y}^{n-1}$	Vector de saída da camada anterior $n-1$
α	Taxa de aprendizagem

REDE NEURONAL

Perda de saída da rede

1. delta_saída$(\boldsymbol{y}^N, \boldsymbol{y})$:
2. return $[\ y_k^N - y_k$ for $k = 1\ to\ |\boldsymbol{y}|\ ]$

$\boldsymbol{y}$	Vector de treino de saída
$\boldsymbol{y}^N$	Vector de saída gerado pela rede
$\|\boldsymbol{y}\|$	Dimensão do vector de treino de saída

REDE NEURONAL

Retropropagar erro pelas camadas da rede, excepto a camada de entrada

1. retropropagar($\boldsymbol{\delta}^N, \alpha$):
2. $\quad \boldsymbol{\delta}^n = \boldsymbol{\delta}^N$
3. $\quad$ for $n = N$ to 2 step -1: — Camada de entrada apenas mantém entradas
4. $\quad\quad \boldsymbol{y}^{n-1} = camadas[n-1].\boldsymbol{y}$
5. $\quad\quad d^{n-1} = camadas[n-1].d_s$ — Dimensão da camada *n* - 1
6. $\quad\quad d^n = camadas[n].d_s$ — Dimensão da camada *n*
7. $\quad\quad neur^n = camadas[n].neuronios$ — Neurónios da camada *n*
8. $\quad\quad \boldsymbol{\delta}^{n-1} = \left[\sum_{j=1}^{d^n} neur^n[j].w_i * \delta_j^n * neur^n[j].y' \text{ for } i = 1 \text{ to } d^{n-1} \right]$
9. $\quad\quad camadas[n].\text{adaptar}(\boldsymbol{\delta}^n, \boldsymbol{y}^{n-1}, \alpha)$
10. $\quad\quad \boldsymbol{\delta}^n = \boldsymbol{\delta}^{n-1}$

$\boldsymbol{\delta}^N$	Vector de variação de erro de saída da rede
$\boldsymbol{\delta}^n$	Vector de variação de erro de saída para cada camada *n*
δ_j	Componente *j* do vector de variação de erro de saída para cada camada
α	Taxa de aprendizagem

REDE NEURONAL

Adaptar parâmetros da rede

1. adaptar($\boldsymbol{x}, \boldsymbol{y}, \alpha$):
2. $\quad \boldsymbol{y}^N = \text{propagar}(\boldsymbol{x})$
3. $\quad \boldsymbol{\delta}^N = \text{delta_saída}(\boldsymbol{y}^N, \boldsymbol{y})$
4. $\quad \text{retropropagar}(\boldsymbol{\delta}^N, \alpha)$
5. $\quad K = \left|\boldsymbol{\delta}^N\right|$
6. $\quad \varepsilon = \frac{1}{K}\sum_{k=1}^{K}\left(\delta_k^N\right)^2$
7. $\quad \text{return } \varepsilon$

$\boldsymbol{x}$	Vector de treino de entrada
$\boldsymbol{y}$	Vector de treino de saída
$\boldsymbol{y}^N$	Vector de saída gerado pela rede
$\boldsymbol{\delta}^N$	Vector de variação de erro de saída
ε	Erro médio de saída da rede
K	Dimensão do vector de perda
α	Taxa de aprendizagem

REDE NEURONAL

Treinar a rede neuronal

```
1.  treinar(X, Y, n_épocas, ε_max, α):
2.      for n_épocas:
3.          ε = 0                          # ε - erro de treino por época
4.          for x ∈ X, y ∈ Y:
5.              ε_x = adaptar(x, y, α)     # ε_x - erro de treino por entrada x
6.              ε = max(ε, ε_x):
7.          if ε ≤ ε_max:
8.              break
```

$\boldsymbol{X}$	Conjunto de entradas $\boldsymbol{x}$
$\boldsymbol{Y}$	Conjunto de saídas $\boldsymbol{y}$
$n_{épocas}$	Número de épocas de treino
ε_{max}	Erro máximo
α	Taxa de aprendizagem

BIBLIOGRAFIA

[Aggarwal, 2018]
C. Aggarwal, Neural Networks and Deep Learning
Springer, 2018

[Munakata, 1998]
T. Munakata, *Fundamentals of the New Artificial Intelligence*, Springer, 1998

[Winston, 1992]
P. Winston, *Artificial Intelligence*, 3rd Edition, Addison-Wesley, 1992

[Raizer *et al.*, 2009]
K. Raizer, H. Idagawa, E. Nobrega, L. Ferreira, *Training and Applying a Feedforward Multilayer Neural Network in GPU*, CILAMCE, 2009

[Gutierrez-Osuna, 2005]
R. Gutierrez-Osuna*, Introduction to Pattern Analysis*, Texas A&M University, 2005

[LeCun *et al.*, 2015]
Y. LeCun, Y. Bengio, G. Hinton, *Deep learning*, Nature, Vol. 521, p. 436-444, 2015

[Deisenroth *et al*., 2020]
M. Deisenroth, A. Faisal, Cheng Soon Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020